

MIP_GITHUB_COPILOT_STRATEGY.md

Mainframe Intelligence Platform

GitHub Copilot Engineering Strategy

Version: 1.0

Status: Approved

Owner: MIP Platform Engineering Team

Executive Summary

GitHub Copilot is a force multiplier for MIP.

However, Copilot is not a developer.

Copilot is not an architect.

Copilot is not the source of truth.

The purpose of this strategy is to define how GitHub Copilot is used consistently across MIP to accelerate engineering while maintaining quality, accuracy, and architectural integrity.

This document establishes:

- Copilot Operating Model
 - Skill Framework
 - Prompt Framework
 - Review Framework
 - Multi-Agent Collaboration Model
 - AI-Assisted Engineering Workflow
-

Core Principle

Source Code Is Truth

Metadata Is Understanding

Knowledge Graph Is Intelligence

AI Is Acceleration

The sequence is:

Source Code

↓

Metadata

↓

Relationships

↓

Knowledge Graph

↓

Reasoning

↓

Modernization Intelligence

↓

AI Assistance

AI never replaces source code analysis.

Why Most Copilot Projects Fail

Most teams use Copilot as:

Question

↓

Answer

↓

Copy Paste

This produces:

- Inconsistent Architecture
- Inconsistent Design
- Hallucinated Decisions
- Technical Debt

MIP does not use Copilot this way.

MIP Copilot Operating Model

Copilot acts as a collection of specialists.

Not a single assistant.

We create reusable engineering personas called Skills.

Skill-Based Engineering

Every engineering task must use a specific skill.

Examples:

Repository Design

↓

repository-engineer

Metadata Modeling

↓

metadata-modeler

Call Graph Design

↓

graph-engineer

Testing

↓

test-engineer

Architecture Review

↓

code-reviewer

Required Skills

Every workspace must install:

```
repository-engineer  
mainframe-modernization-architect  
mainframe-code-analyst  
metadata-modeler  
graph-engineer  
sqlite-engineer  
test-engineer  
code-reviewer  
documentation-writer
```

Why Skills Matter

Without skills:

Different engineers ask different questions.

Different answers emerge.

Architecture drifts.

With skills:

Consistent behavior emerges.

Copilot Workflow

Never ask Copilot for code first.

Always follow:

Design

↓

Review

↓

Critique

↓

Improve

↓

Implement

↓

Test

↓

Refactor

↓

Document

Design First Development

Before writing code:

Ask Copilot:

Design the architecture.

Do not generate code.

Identify:

- Components
- Responsibilities
- Risks

- Dependencies
- Testing Strategy

Only after approval:

Generate implementation.

Review Before Implementation

Every design must be reviewed.

Use:

code-reviewer

Prompt:

Review this design.

Identify:

- Architectural Risks
- Scalability Risks
- Testing Risks
- Maintainability Risks

Do not rewrite.

Critique only.

Critique Driven Development

MIP encourages criticism before implementation.

Goal:

Find mistakes while they are cheap.

Not after deployment.

Prompt Engineering Standard

Every prompt must contain:

Role

Context

Requirements

Output Format

Constraints

Review Questions

Example Prompt Template

Act as a Senior Software Architect.

Context:

Building metadata extraction platform.

Requirements:

- Python
- SQLite
- NetworkX

Output:

Architecture only.

Constraints:

No code.

Review:

Identify risks.

Copilot Modes

MIP uses four modes.

Mode 1

Architect Mode

Purpose:

Design systems.

Questions:

What should we build?

How should components interact?

What are the risks?

Skills:

- repository-engineer
 - mainframe-modernization-architect
-

Mode 2

Analyst Mode

Purpose:

Understand legacy systems.

Questions:

What does this program do?

What does this job execute?

What tables are accessed?

Skills:

- mainframe-code-analyst
-

Mode 3

Engineer Mode

Purpose:

Implement solutions.

Questions:

Generate implementation.

Generate tests.

Generate models.

Skills:

- metadata-modeler
 - sqlite-engineer
 - graph-engineer
-

Mode 4

Reviewer Mode

Purpose:

Critique implementation.

Questions:

What is wrong?

What can fail?

What should improve?

Skills:

- code-reviewer
 - test-engineer
-

Multi-Agent Collaboration Pattern

MIP treats skills as specialist agents.

Example:

Repository Design

Step 1

repository-engineer

Creates structure.



Step 2

code-reviewer

Reviews structure.



Step 3

documentation-writer

Documents structure.

This produces higher quality outcomes.

Copilot Usage Rules

Rule 1

Never ask for implementation before design.

Rule 2

Always review generated designs.

Rule 3

Always review generated code.

Rule 4

Always generate tests.

Rule 5

Always document decisions.

MIP Development Pattern

Phase 1

Discovery

Copilot assists inventory generation.

Phase 2

Metadata Extraction

Copilot assists parser design.

Phase 3

Relationship Discovery

Copilot assists graph design.

Phase 4

Knowledge Graph

Copilot assists graph traversal design.

Phase 5

Reasoning

Copilot assists impact analysis.

Phase 6

Modernization

Copilot assists transformation planning.

Copilot Anti-Patterns

Avoid:

Generating entire systems without architecture.

Blindly accepting generated code.

Skipping testing.

Skipping review.

Treating AI output as truth.

Using a single prompt for everything.

Prompt Library Strategy

All prompts must reside in:

```
prompts/  
  
architecture/  
discovery/  
parsing/  
metadata/  
graph/  
testing/  
review/  
modernization/
```

No ad hoc prompts.

Prompts become reusable assets.

Repository Discovery Workflow

Example:

repository_scan.md

↓

Generate inventory

↓

Review inventory

↓

Store metadata

↓

Build graph

↓

Generate insights

Copilot assists every stage.

Code Generation Workflow

Step 1

Design

Step 2

Review

Step 3

Implement

Step 4

Test

Step 5

Review

Step 6

Refactor

Step 7

Document

Test Generation Workflow

Never write implementation without tests.

Ask Copilot:

Generate:

Unit Tests

Integration Tests

Edge Cases

Failure Cases

Performance Tests

Documentation Workflow

Every feature requires:

Purpose

Architecture

Implementation Notes

Testing Notes

Future Improvements

Documentation is not optional.

Copilot Success Metrics

Measure:

- Design Quality
- Test Coverage
- Review Coverage
- Documentation Coverage
- Reuse of Skills
- Reuse of Prompts

Do not measure:

- Lines of code
 - Number of prompts
 - Number of completions
-

Leadership Vision

The goal is not:

"Use Copilot"

The goal is:

"Create an AI-Augmented Engineering System"

Where:

- Knowledge is reusable
 - Prompts are reusable
 - Skills are reusable
 - Decisions are documented
 - Engineering quality improves over time
-

Definition Of Done

Copilot strategy is successfully adopted when:

- ✓ Skills are installed
- ✓ Prompt library exists

- ✓ Reviews occur before implementation
 - ✓ Tests are generated consistently
 - ✓ Documentation is generated consistently
 - ✓ Architecture decisions are documented
 - ✓ Engineering quality improves sprint over sprint
-

Success Metric

A new engineer joins the team and can:

- Understand the architecture
- Discover source code
- Design components
- Implement features
- Generate tests
- Review changes

Using the same skills and prompts as every other engineer.

This creates a repeatable engineering system rather than individual heroics.